Assignment #4: Text Classification, LLM RAG, Agents, and Speech

Total score: 120

This assignment can be completed individually or by a team of a maximum of three members.

Due date: See the course page

Learning Objectives

By completing this assignment, you will preprocess and vectorize text data, train and evaluate a traditional text classifier, compare it with an LLM-based classification agent, build a Retrieval-Augmented Generation (RAG) application, and integrate speech-to-text (STT) and text-to-speech (TTS) capabilities into the application.

Dataset Description and General Instructions

Use the **AG news dataset** contains short news articles labeled with one of four categories:
1: World, 2: Sports, 3: Business, 4: Sci/Tech

Use the official 120,000 example training set and 7,600 example test set splits.

Part 1: Text Data Preprocessing, Vectorization, and Classification (40 points)

Write "**topic_classifier.py**" that runs the required experiments and print or save the key results.

(1) Load, inspect, and vectorize the data

(a) Load the 120,000 training examples and 7,600 test examples using **torchtext**, "torchtext.datasets.AG_NEWS" or an appropriate alternative if the installed version of torchtext does not support it.

(b) Combine each article's title and description into one text string for feature extraction and classification.

(c) Display one example article from each category and the number of examples per category in both splits.

(d) Create a document-term matrix (DTM) with CountVectorizer and a TF-IDF feature matrix with TfidfVectorizer. Fit each vectorizer on training text only, then transform the test text.

(e) Report the shape (number of documents × number of features) of the training DTM. Calculate its sparsity percentage using: $Sparsity\ (\%) = (1 - \frac{Number\ of\ nonzero\ entries}{Total\ number\ of\ entries}) \times 100$

(f) Briefly define document and corpus. Explain what a DTM row, column, and cell represent, and how a DTM cell differs from a TF-IDF cell. Explain why these matrices are sparse and how sparsity affects storage or computation.

(2) Train and evaluate an MLP classifier

Train a Multilayer Perceptron (MLP) using the CountVectorizer features. Report the architecture (hidden layers and units), activation function(s), optimizer, loss function, learning rate, batch size, number of epochs/iterations, and any other non-default settings needed to reproduce the run.

Evaluate on the held-out test set and report:

- Accuracy and macro-averaged precision (state the averaging method).
- A confusion matrix with the four category names.
- Three misclassified test examples. For each, show the true category, predicted category, and a brief plausible explanation of the error.

Why documents, not terms, are the observations for supervised document classification, and why transposing the DTM changes the prediction target rather than producing the same task.

(3) Compare feature representations and stop-word removal

Run the four configuration below. Use the same train/test split, MLP architecture, and training settings for all four runs. Fit each vectorize on training text only. Report test accuracy and macro-precision for each configuration in **one table** that summarizes the results, including metrics and one confusion matrix.

Vectorizer configuration	*Stop-word removal*
CountVectorizer	No
TfidfVectorizer	No
CountVectorizer	Yes
TfidfVectorizer	Yes

Briefly discuss whether TF-IDF and stop-word removal changed performance and offer a reasonable explanation based on your results.

(4) Sentiment analysis

In about 100 words, outline how you would build a sentiment-analysis pipeline for Business news. Include data preparation, sentiment labels, feature extraction, and evaluation. Explain how sentiment analysis differs from the topic classification task in this assignment.

Part 2. LLM-based Classification Agents (20 points)

Create "**classification_agent.py**" using the **LangChain agent framework** and the LLM "**gpt-oss**" model on the Nautilus Hypercluster.

(1) Implement the agent

(a) Accept a single AG News article and classify it as World, Sports, Business, or Sci/Tech.

(b) Use zero-shot instructions. Show the prompt or prompt template in your report.

(c) Return valid JSON with exactly these fields: category and justification. The category must be one of the four permitted category names; the justification should be brief.

(d) Validate the returned JSON and category. Handle invalid output gracefully (for example, retry once or return a clear validation error). Do not include hidden chain-of-thought; provide only a concise explanation of the classification.

(2) Compare with the Part 1 classifier

Evaluate the agent and your best Part 1 classifier on the same stratified sample of 200 examples from the official test set. Use random seed 42 for sample selection and report the sample size. If your environment cannot evaluate 200 examples, explain why, use at least 50 examples, and state the limitation. Do not use these examples to tune the Part 1 model.

Present one comparison table that includes:

- Accuracy for each method on the same examples.
- Average agent latency per example and total evaluation time. If using a paid API, report the cost if available; if not, say how cost was estimated or why it is unavailable. For a local model, report the model and relevant hardware/resource requirements.
- A brief comparison of zero-/few-shot data efficiency versus training on 120,000 labeled examples.
- A short comparison of the usefulness and limitations of the agent's justifications and the traditional model's error analysis.

Create **one comparison table** that summarizes the results, including accuracy, latency, cost/resources, and limitations.

Part 3. Retrieval-Augmented Generation (RAG) Application (30 points)

Use the "gpt-oss" model on the Nautilus Hypercluster, following the course instructions for environment setup and model access. The knowledge source must be the course syllabus PDF, "CPSC 254_SYL.pdf", posted on Canvas.

(1) Implement the RAG pipeline

Create "**rag.py**". It must:

(a) Load the syllabus PDF and extract its text.
(b) Split the text into overlapping chunks. Document the chunk size and overlap.
(c) Create embeddings for the chunks and store them, together with their source text, in a searchable **Chroma** vector database.
(d) Retrieve relevant chunks for a user question and pass the retrieved text and question to gpt-oss to generate an answer grounded in the syllabus.
(e) Save the vector index and any required configuration so subsequent runs can reuse them instead of rebuilding the index unnecessarily.
(f) Make it clear when the syllabus does not contain enough information to answer; do not invent syllabus details.

(2) Evaluate the RAG application

Create "**rag_eval.py**". It must load the saved index and evaluate at least three syllabus-specific questions. Choose questions whose answers can be verified directly from the syllabus, including at least one question that requires locating a specific requirement, date, policy, or course detail.

For each question, include in the report:

- The exact question and the generated answer.
- The supporting syllabus text or section/page reference.
- A rating of accuracy, completeness, and relevance: each rated 0 (poor), 1 (partly correct), or 2 (good), with one brief justification.

- Any unsupported claims, errors, or important omissions.

Include one screenshot or captured terminal output showing a successful evaluation run with the questions and answers visible.

Create **one table** that summarizes the results, including three questions, supporting evidence, answers and ratings.

Part 4. Speech Recognition and Voice Synthesis (20 points)

Extend the Part 3 RAG application using the course-provided STT/TTS examples. You may use microphone input or load a provided audio file, as supported by your implementation.

(1) Speech-to-text (STT)

Create "**stt_rag.py**". Capture or load spoken input, transcribe it with an STT system, send the transcribed question to the RAG application, and display both the transcription and the generated answer.

(2) Text-to-speech (TTS)

Create "**tts_rag.py**". Send a question through the RAG application, convert the resulting text answer to speech, and play the audio aloud. The report must demonstrate the end-to-end flow from question to spoken answer.

Include **screenshots** or other appropriate evidence for both programs. If microphone or audio playback is not available in your execution environment, document the limitation and provide evidence using a supported audio-file workflow.

Submission Requirements

Report (10 points)

Submit one report per individual or team in Microsoft Word or PDF format. Include:

- Names of all team members and each member's percentage contribution. If contributions were equal, state "Equal contribution." Otherwise, summarize each member's assigned and completed tasks. Individual grades may differ based on each member's contribution.

- Results and explanations requested in Parts 1–4, including tables, metrics, error analysis, and RAG answer evaluations.
- Evidence that each required program runs successfully. Screenshots should show useful output, not only an editor window or code.
- A brief summary of the main findings, limitations, and any deviations from the requirements.

Program files

Submit the following six Python source files:

- topic_classifier.py
- classification_agent.py
- rag.py
- rag_eval.py
- stt_rag.py
- tts_rag.py

Include any configuration or supporting files you created that are necessary to run the programs. Do not submit the original dataset or course-provided sample programs. Do not include API keys, passwords, or other secrets. Include a short README with run instructions, dependencies, environment assumptions, and the expected command for running each program.

Grading Criteria

Your work will be evaluated based on the following criteria:

- **Functional correctness**: Do the programs successfully perform the required tasks and produce valid, appropriate results?
- **Conceptual understanding**: Does the report demonstrate a clear understanding of the relevant concepts, algorithms, programming techniques, and system requirements?
- **Experimental design and evaluation**: Are the models evaluated appropriately, with suitable metrics, fair comparisons, and meaningful error analysis?
- **Implementation quality**: Are the programs organized, reproducible, and consistent with the assignment requirements?

- **Analysis and documentation**: Does the report explain the results, discuss limitations, and provide appropriate evidence of successful implementation?
- **Effort and contribution**: Is the work reflected in the quality, completeness, and depth of the report and program files?